

EXPERIMENT 06 – WINDOWS, TABS, AND IFRAMES

1. Module Name and Experiment Title

Module Name: Module 1 – Automation with Selenium

Experiment Number: 06

Experiment Title: Windows, Tabs, and Iframes

Programming Language: Python

Website 1: DemoQA – Browser Windows

Website URL: <https://demoqa.com/browser-windows>

Website 2: Automation Testing – Frames

Website URL: <https://demo.automationtesting.in/Frames.html>

2. Problem Statement

Modern web applications may open new browser tabs or windows and display content inside iframes. Selenium WebDriver initially interacts with the current browser window and cannot directly access elements inside an iframe without switching to it.

The objective of this experiment is to develop a Selenium automation script that can open and switch between browser tabs and windows, interact with elements inside an iframe, and return to the main page.

The script should also capture screenshots of the automation process and close the browser automatically after completing all operations.

3. Objective

The main objectives of this experiment are:

1. To understand how Selenium handles multiple browser windows and tabs.
 2. To learn how to switch between browser windows using window handles.
 3. To understand the concept and purpose of iframes.
 4. To switch into an iframe and interact with its elements.
 5. To return to the main page after interacting with an iframe.
 6. To capture screenshots during the automation process.
 7. To execute the complete automation workflow and close the browser automatically.
-

4. Tools, Software and Concepts Used

Tools and Software

- **Programming Language:** Python
- **Automation Tool:** Selenium WebDriver

- **Browser:** Google Chrome
- **Driver:** ChromeDriver
- **Code Editor:** Visual Studio Code

Websites Used

- **DemoQA – Browser Windows:** Used to demonstrate opening and switching between browser tabs and windows.
- **Automation Testing – Frames:** Used to demonstrate switching into an iframe and interacting with its input field.

Concepts Used

- Selenium WebDriver
- Browser window handles
- Window switching
- Tab switching
- Iframe handling
- Explicit waits
- `switch_to.window()`
- `switch_to.frame()`
- `switch_to.default_content()`
- Screenshot capture
- Element location using ID and CSS selectors

5. Source Code / Implementation Steps

Source Code

File Name: exp_06_windows_tabs_iframes.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Initialize WebDriver
driver = webdriver.Chrome()
wait = WebDriverWait(driver, 15)

try:
    driver.maximize_window()

    # Step 1: Open the main browser windows page
    driver.get("https://demoqa.com/browser-windows")
    main_window = driver.current_window_handle
```

```

print("Main window opened successfully.")
time.sleep(2)

# Step 2: Open and switch to a new tab
wait.until(
    EC.element_to_be_clickable((By.ID, "tabButton"))
).click()

wait.until(lambda d: len(d.window_handles) == 2)

for handle in driver.window_handles:
    if handle != main_window:
        driver.switch_to.window(handle)
        break

print("Switched to new tab.")
print("Tab title:", driver.title)

driver.save_screenshot("screenshots/01_new_tab.png")
print("Screenshot 1: New tab saved.")

driver.close()
driver.switch_to.window(main_window)
print("Returned to main window.")

# Step 3: Open and switch to a new window
wait.until(
    EC.element_to_be_clickable((By.ID, "windowButton"))
).click()

wait.until(lambda d: len(d.window_handles) == 2)

for handle in driver.window_handles:
    if handle != main_window:
        driver.switch_to.window(handle)
        break

print("Switched to new window.")
print("Window title:", driver.title)

driver.save_screenshot("screenshots/02_new_window.png")
print("Screenshot 2: New window saved.")

driver.close()
driver.switch_to.window(main_window)
print("Returned to main window.")

# Step 4: Open the iframe demonstration page
driver.get("https://demo.automationtesting.in/Frames.html")

# Switch to the single iframe
iframe = wait.until(
    EC.presence_of_element_located((By.ID, "singleframe"))
)

driver.switch_to.frame(iframe)
print("Switched to iframe successfully.")

# Step 5: Interact with the iframe input field
input_field = wait.until(

```

```

        EC.visibility_of_element_located(
            By.CSS_SELECTOR, "input[type='text']")
    )

    input_field.send_keys("Selenium iframe automation")
    print("Text entered into iframe successfully.")

    time.sleep(2)
    driver.save_screenshot("screenshots/03_iframe_content.png")
    print("Screenshot 3: Iframe content saved.")

    # Step 6: Return to the main page
    driver.switch_to.default_content()
    print("Returned to main page.")

    driver.save_screenshot("screenshots/04_main_page.png")
    print("Screenshot 4: Main page saved.")

    print("Windows, tabs, and iframe handling successful!")

finally:
    driver.quit()
    print("Browser closed successfully.")

```

Implementation Steps

Step 1: Initialize the WebDriver

Import the required Selenium modules, create a Chrome WebDriver instance, and configure an explicit wait of 15 seconds.

Step 2: Open the Main Browser Window

Navigate to the DemoQA Browser Windows page and store the current window handle. This handle will be used to return to the original browser window.

Step 3: Open and Switch to a New Tab

Click the New Tab button and wait until the new tab appears. Use the window handles to switch to it, capture a screenshot, and close the tab.

Step 4: Return to the Main Window

Switch back to the original browser window using its stored handle.

Step 5: Open and Switch to a New Window

Click the New Window button and wait for the new window to open. Switch to it, capture a screenshot, and close it. Return to the original window afterward.

Step 6: Open the Iframe Demonstration Page

Navigate to the Automation Testing Frames page and locate the single iframe using its ID, `singleframe`.

Step 7: Switch into the Iframe

Use `switch_to.frame()` to switch the driver's focus from the main page to the iframe.

Step 8: Interact with the Iframe

Locate the text input field inside the iframe and enter the text `Selenium iframe automation`. Capture a screenshot showing the entered text.

Step 9: Return to the Main Page

Use `switch_to.default_content()` to leave the iframe and return to the main page. Capture a screenshot of the page.

Step 10: Close the Browser

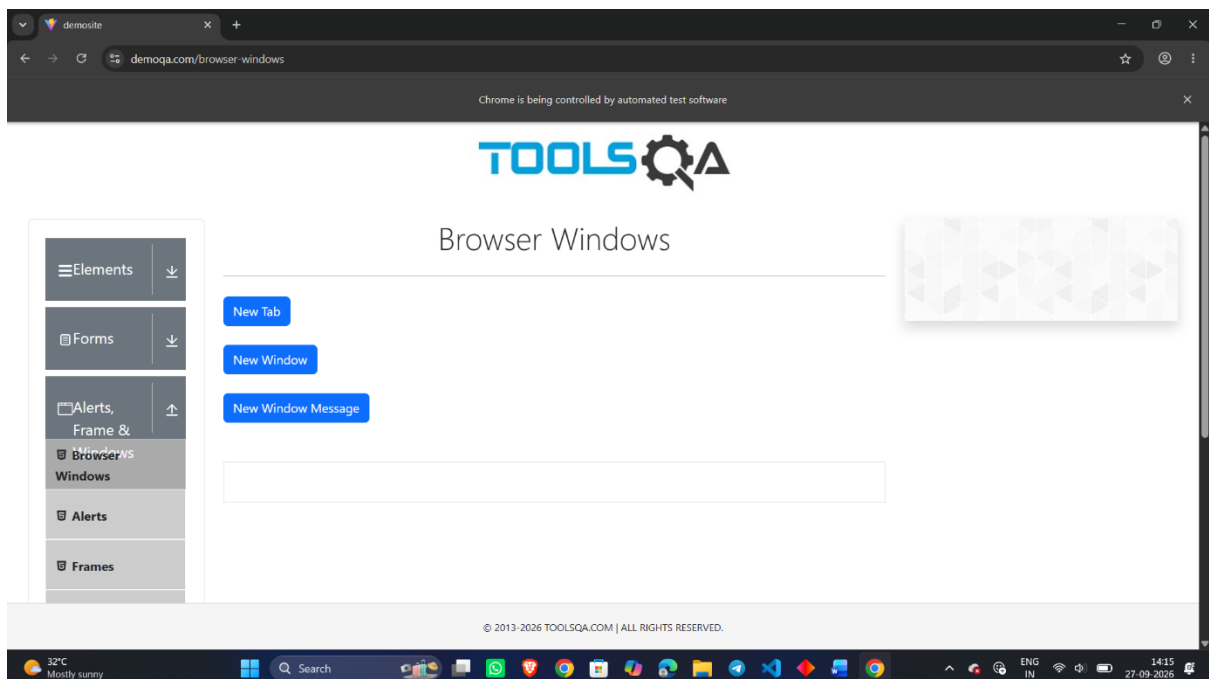
Use `driver.quit()` inside the `finally` block to ensure that the browser closes after the automation process finishes.

6. Output with Clearly Labelled Screenshots

The following screenshots demonstrate the different stages of the experiment. Insert the corresponding screenshots captured during execution.

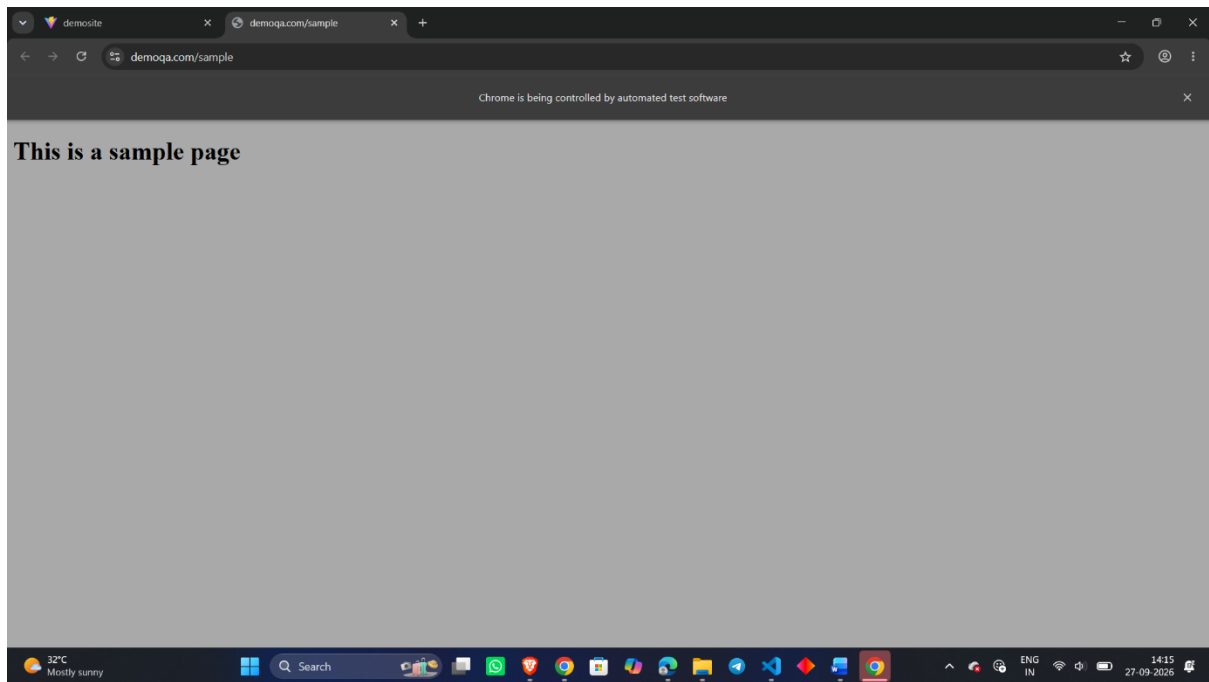
Screenshot 1: Main Browser Window

Description: This screenshot shows the initial DemoQA Browser Windows page after Selenium opens it.



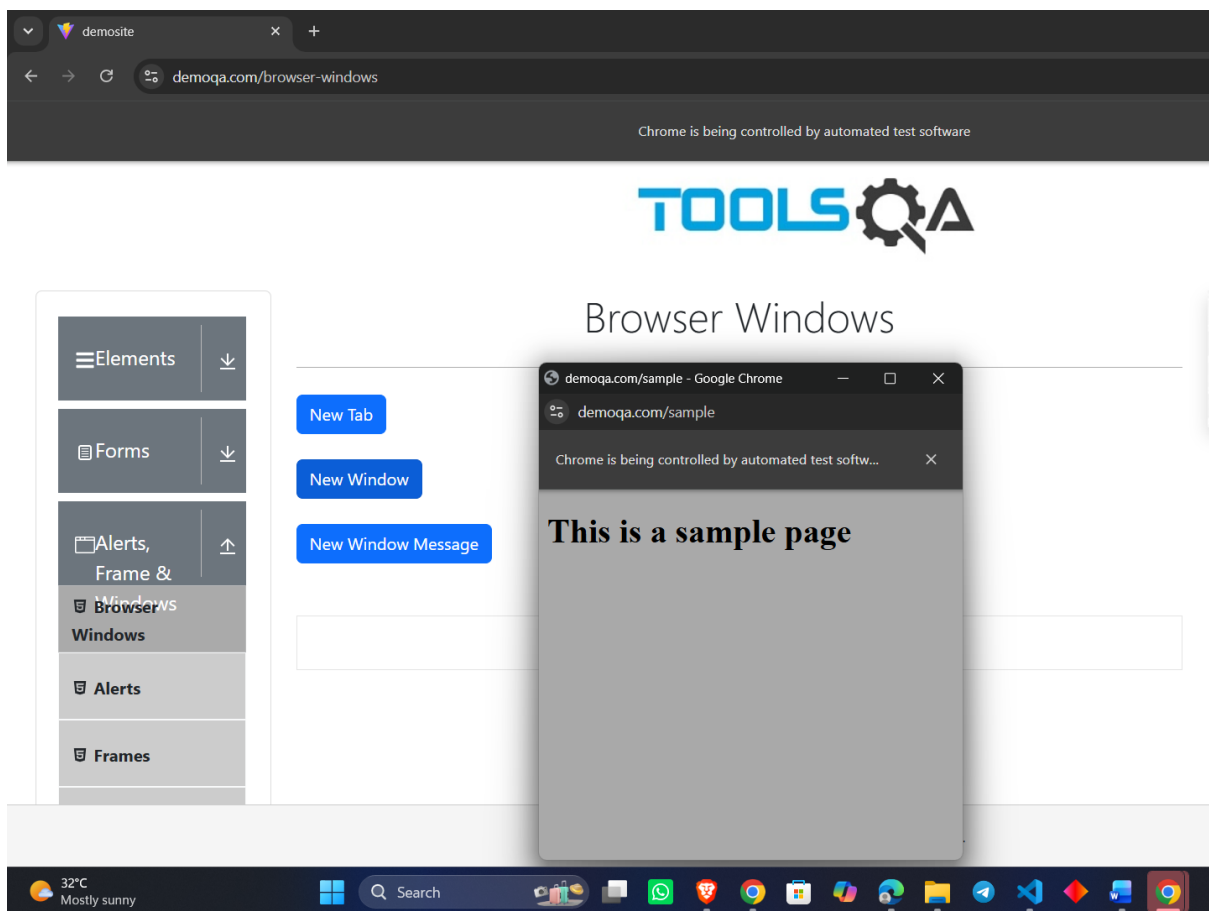
Screenshot 2: New Tab Handling

Description: This screenshot shows the newly opened browser tab after Selenium switches to it.



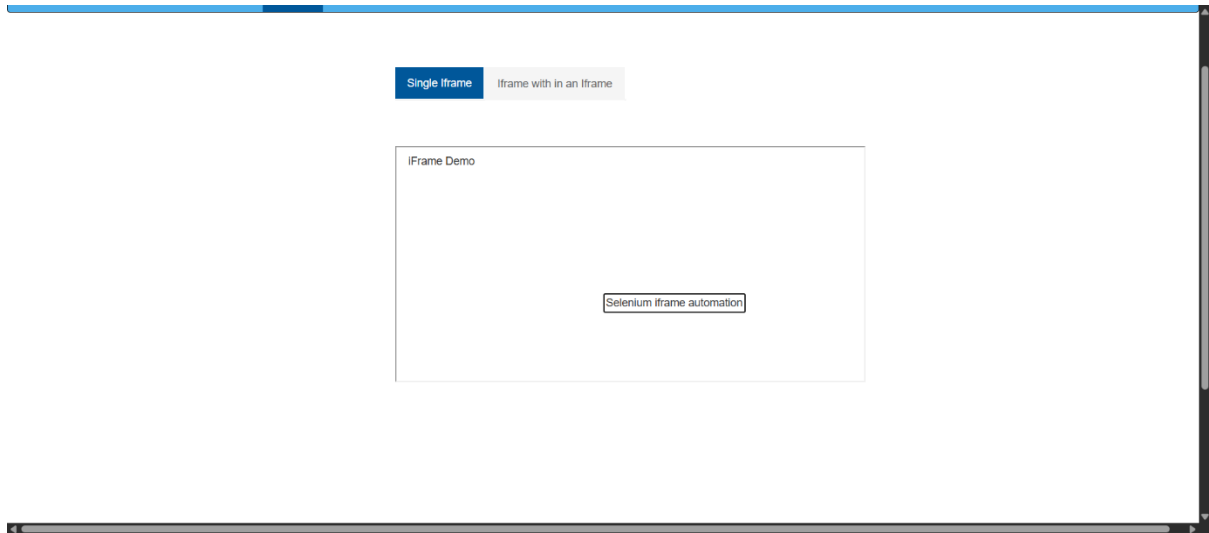
Screenshot 3: New Window Handling

Description: This screenshot shows the newly opened browser window after Selenium switches to it.



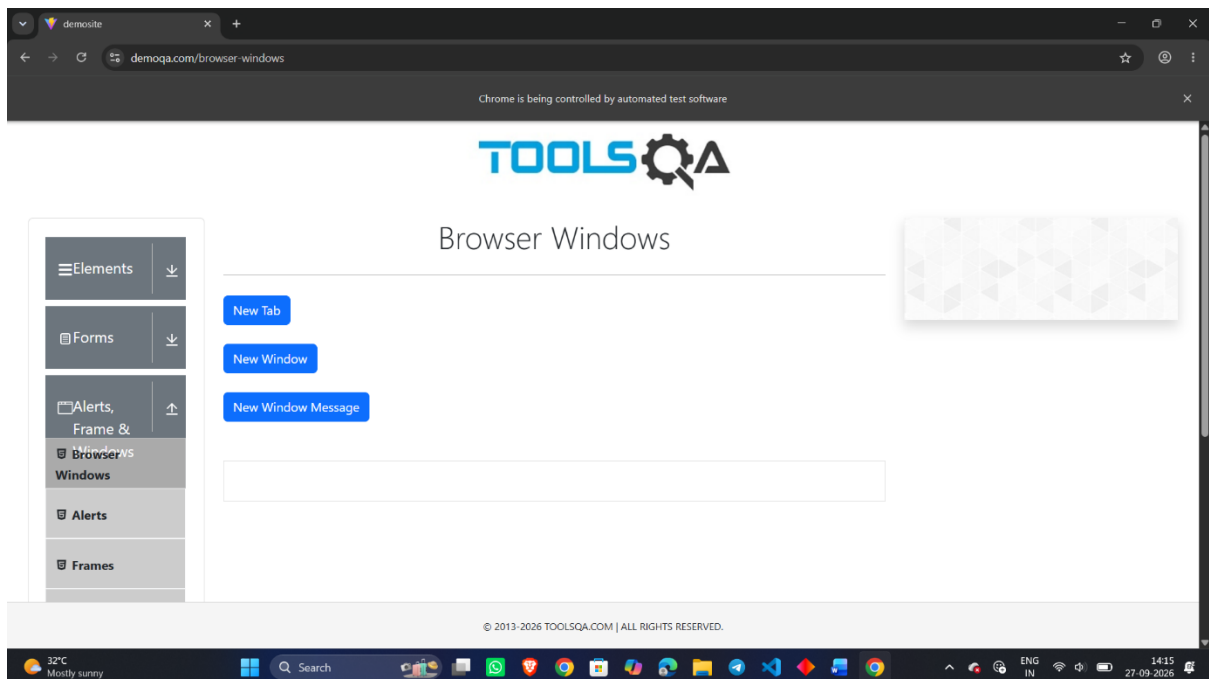
Screenshot 4: Iframe Interaction

Description: This screenshot shows the Automation Testing Frames page after Selenium switches into the single iframe and enters the text `Selenium iframe automation`.



Screenshot 5: Return to Main Page

Description: This screenshot shows the main Frames page after Selenium exits the iframe using `switch_to.default_content()`.

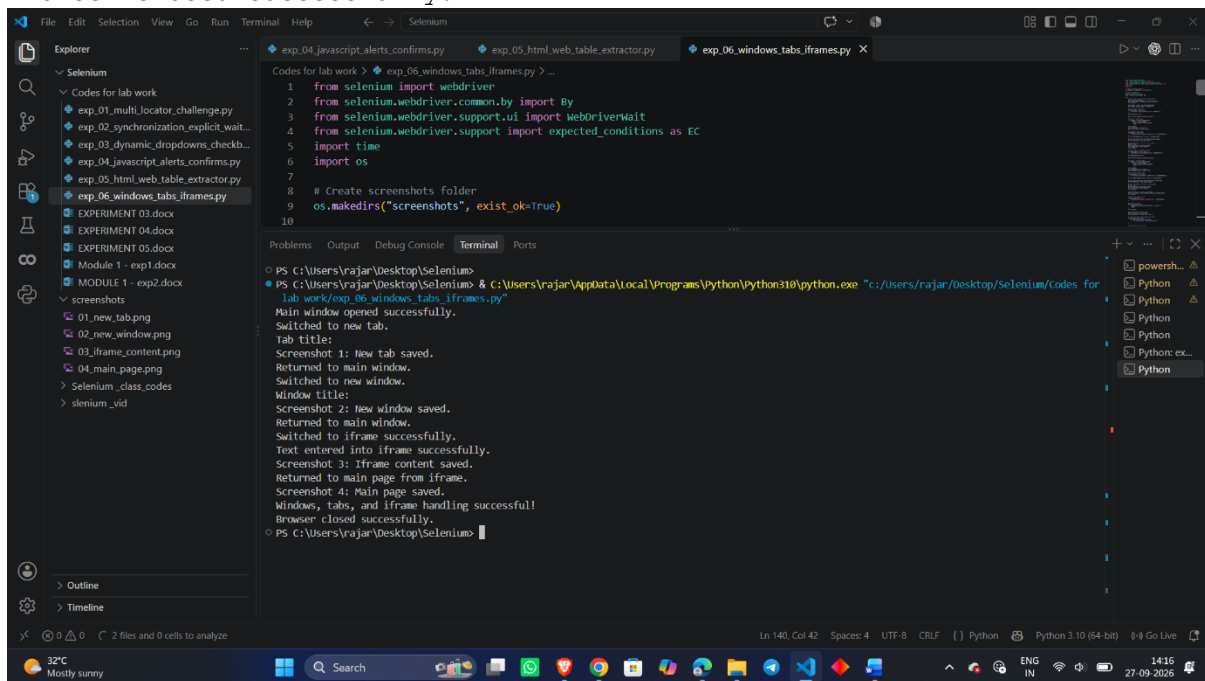


Console Output

The successful execution produces output similar to the following:

```
Main window opened successfully.
```

Switched to new tab.
Tab title: ...
Screenshot 1: New tab saved.
Returned to main window.
Switched to new window.
Window title: ...
Screenshot 2: New window saved.
Returned to main window.
Switched to iframe successfully.
Text entered into iframe successfully.
Screenshot 3: Iframe content saved.
Returned to main page.
Screenshot 4: Main page saved.
Windows, tabs, and iframe handling successful!
Browser closed successfully.



7. Result, Observation and Conclusion

Result

The experiment demonstrated the handling of browser tabs, windows, and iframes using Selenium WebDriver. The automation script opened and switched between browser contexts, interacted with an input field inside an iframe, returned to the main page, captured screenshots, and closed the browser automatically.

Observation

1. Selenium uses window handles to identify and switch between browser windows and tabs.
2. A newly opened tab or window requires switching to its corresponding window handle before interacting with it.
3. Elements inside an iframe can be accessed after switching into that iframe.
4. The `switch_to.default_content()` method returns control to the main page.

5. Explicit waits help ensure that elements are available before interaction.
6. The `driver.quit()` method closes the browser and all associated windows.

Conclusion

This experiment provided practical experience in handling multiple browser windows, tabs, and iframes using Selenium WebDriver. It demonstrated how to switch between different browser contexts, interact with elements inside an iframe, and return to the main page.

The experiment also highlighted the importance of window handles and frame switching in web automation. These techniques are useful when automating web applications that contain multiple browser windows, tabs, or embedded content.